

UNIVERSIDADE FEDERAL DE RORAIMA – CAMPUS PARICARANA
CURSO DE CIÊNCIA DA COMPUTAÇÃO

APOLO MATOS PEDROSO

IVAN MEDINA MAIA

Relatório técnico do Laboratório de Circuitos

BOA VISTA – 2026

APOLO MATOS PEDROSO

IVAN MEDINA MAIA

Relatório técnico do Laboratório de Circuitos

Relatório técnico do Laboratório de Circuito
como requisito para compor a nota da média da
disciplina Arquitetura e Organização de
Computadores DCC301 do segundo semestre
de 2026.

Prof. Dr. Herbert Oliveira Rocha

1 INTRODUÇÃO

O Laboratório de Circuitos tem como objetivos principais desenvolver (1) um módulo de memória de 64 bytes com decodificação de endereços e uma cache de 4 linhas medida sobre um traço de leitura; e (2) uma unidade de controle cabeada de 5 estados e o microprograma equivalente, a partir de um lista de circuitos que foi dado previamente. O laboratório foi feito utilizando o GitHub e o software de simulação Logisim, visando entender a montagem de um hardware e a análise de seus comportamentos durante a simulação com tabelas.

2 DESENVOLVIMENTO

2.1 PARTE UM

2.1.1 MÓDULOS DE MEMÓRIA

A parte um do laboratório é a construção de um espaço de memória de 64 bytes, dividido de quatro regiões de 16 bytes, com um decodificador escolhendo qual delas responde a cada endereço. A tabela abaixo mostra a tabela verdade que determina o módulo de memória que tem acesso ao barramento de dados selecionado através dos bits A5 e A4.

A5	A4	Região da memória
0	0	CS_ROM
0	1	CS_RAM
1	0	CS_REG
1	1	CS_I/O

Tabela 1: tabela verdade dos sinais de seleção da região dos módulos de memória.

Para definir qual região poderá enviar dados para o barramento de dados, todos os 4 bits A4, A3, A2 e A1 entram no multiplexador 4:1, com os bits A5 e A4 usados para selecionar a região do módulo de memória.

A figura abaixo mostra o datapath da região do módulos de memória, contendo o circuito para a realização do cálculo do bit de paridade ímpar e sua verificação, além de também mostrar como está organizado barramentos de endereço, entrada e saída de dados.

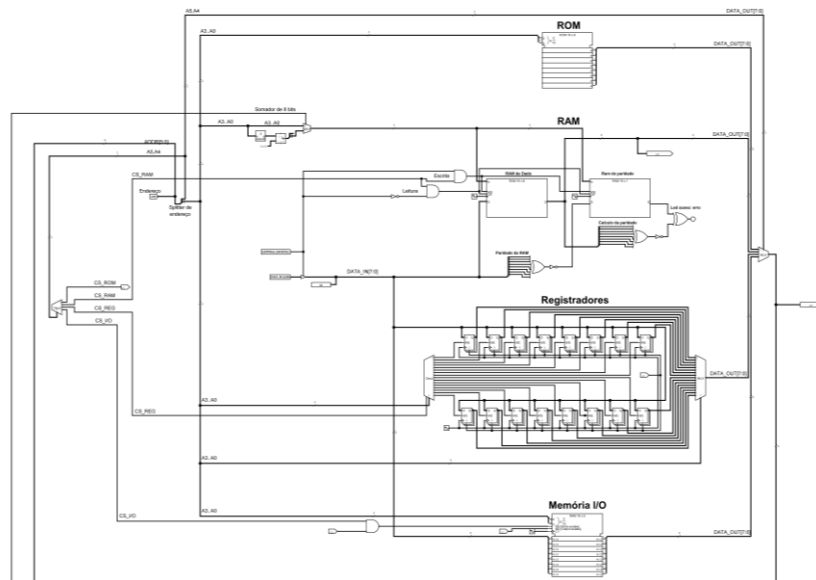


Figura 1: datapath para os módulos de memória.

Vale o destaque na figura abaixo mostrando a memória RAM junto ao circuito que realiza o cálculo de paridade ímpar e a sua verificação no momento da leitura, para a construção do circuito de cálculo de paridade, foram utilizadas portas XOR com 8 bits de entrada, e Splitters para a divisão dos bits. Para a verificação do erro de calculo do bit de paridade, foi necessário forçar uma gravação de outro valor diferente do que o cálculo faria para poder exibir a mensagem de erro, destacado pelo LED vermelho aceso.

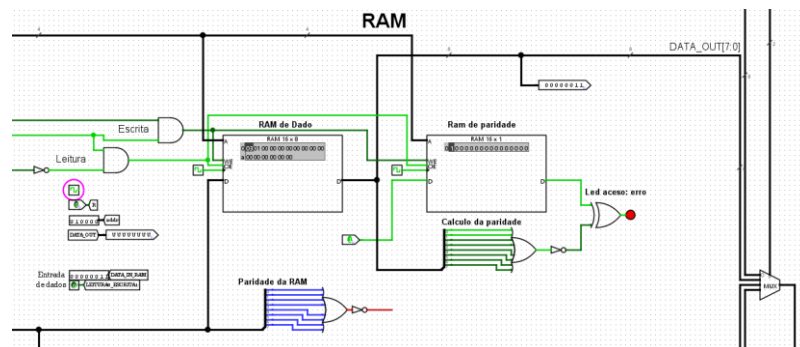


Figura 2: Detecção do erro do cálculo do bit de paridade.

2.1.2 CACHE

A figura abaixo mostra a construção do circuito da linha 0 da cache, como foi montado o circuito para a comparação das TAGs, utilização portas discretas como a NOT, a OR e a AND.

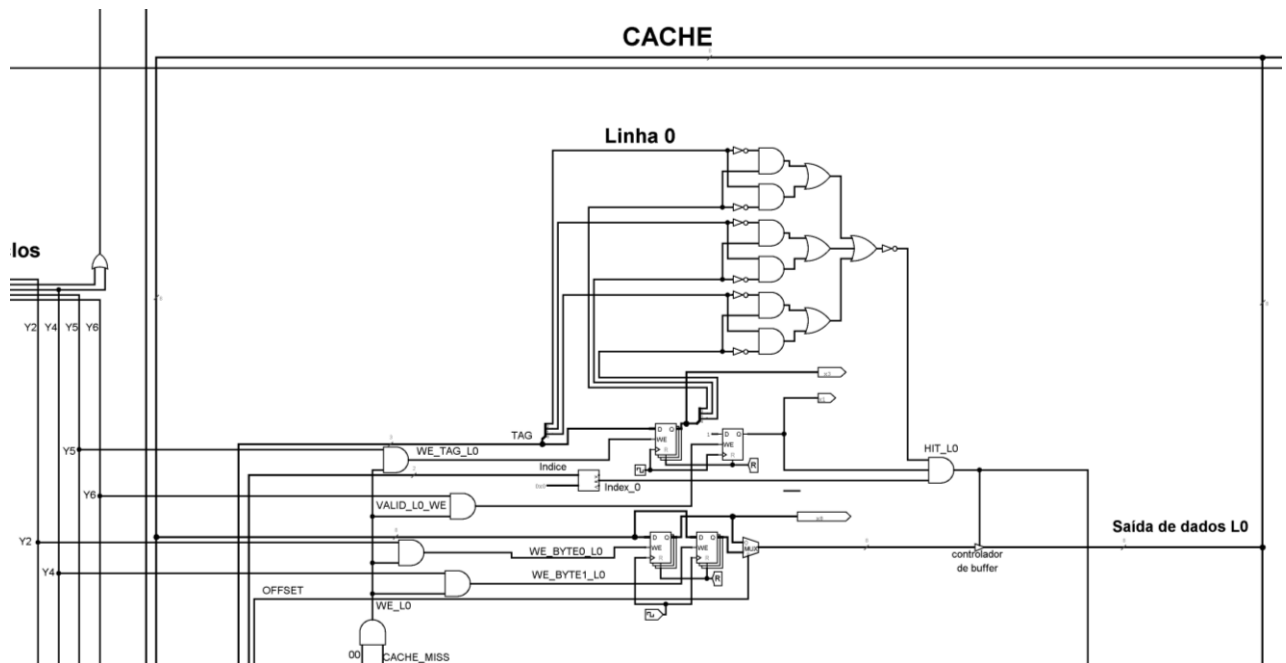


Figura 3: A linha 0 da cache com o circuito comparador de TAG.

Figura abaixo mostra o datapath completo da cache do tipo leitura para o circuito, composto por 4 linhas, com bloco de 2 bytes. O também é apresentado o circuito para a comparação de TAGs, circuito para contagem de ciclos e contagem de acertos. Neste cache, o barramento de endereço é composto por 6bits, sendo de A5..A0. Caminho de dados é determinado pela divisão desses bits, sendo os 3 mais significativos para determinar a Tag, A5, A4 e A3, o bits do meio, A2 e A1 para determinar a linha e o bit menos significativo para escolher o byte daquele bloco, dividido em bloco 0 e bloco 1.

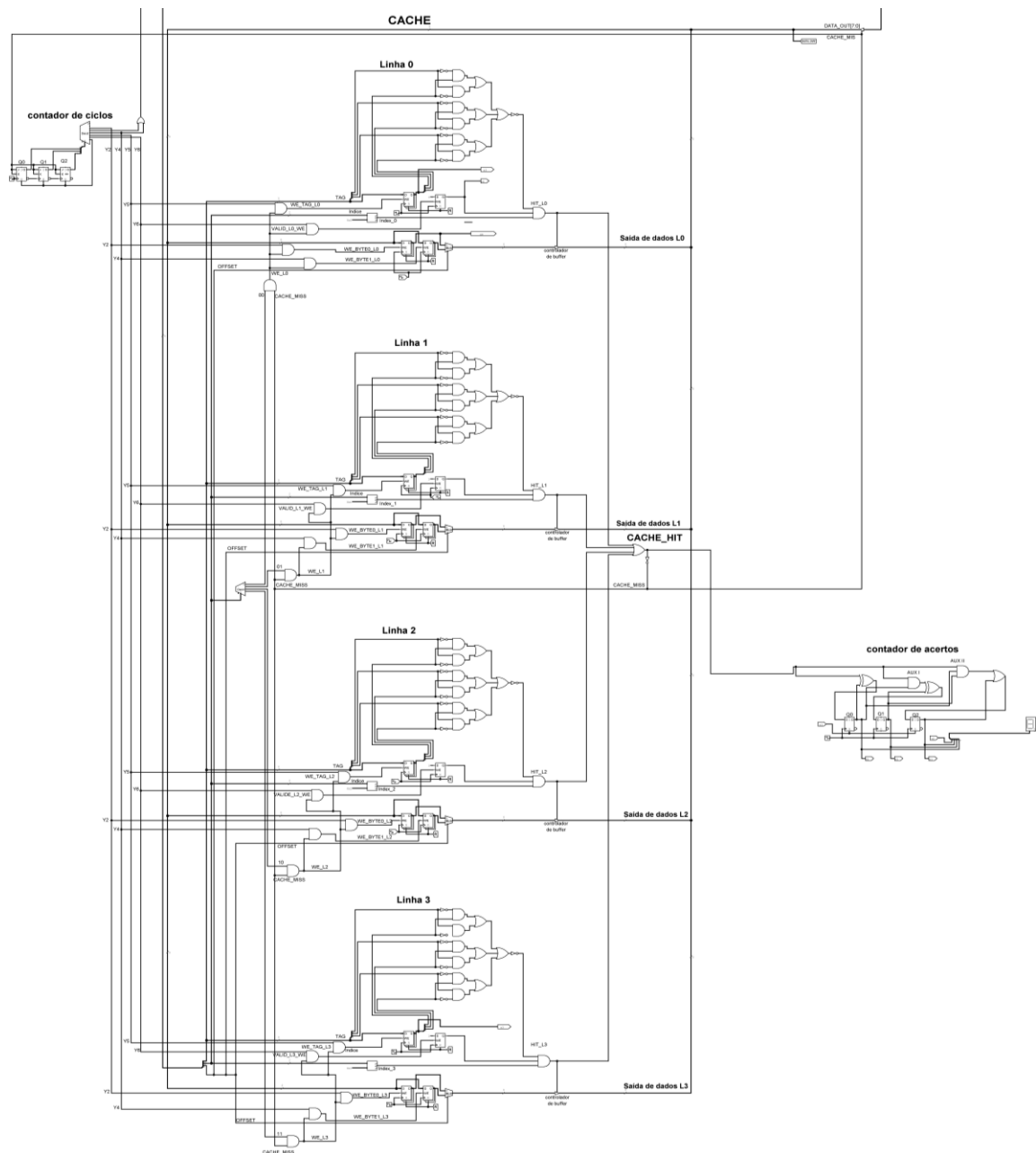


Figura 4: Demonstração do circuito da cache com seus respectivos componentes.

2.2 PARTE DOIS

Na parte dois, é pedido a construção de dois circuitos de arquitetura MIPS multiciclo, uma com unidade de controle cabeada e outra com unidade de controle microprogramada.

Para ambos os circuitos adota-se a arquitetura MIPS multiciclo reduzido a 8 bits, e a sua ISA é de apenas dois tipos, Tipo-R com as instruções “add”, “sub”, “and” e “or” e do Tipo-I(desvio) com apenas a instrução “beq”. Como cada instrução possui 8 bits e não há instruções que necessitam do campo “shamt”, a divisão da instrução fica da seguinte forma:

Campo da instrução	Bits
Opcode	2 ([7:6])
Rs	2 ([5:4])
Rt	2 ([3:2])
Funct/Rd/Imediato	2 ([1:0])

Tabela 2: Divisão de cada instrução.

A Imagem abaixo mostra o datapath de ambos circuitos:

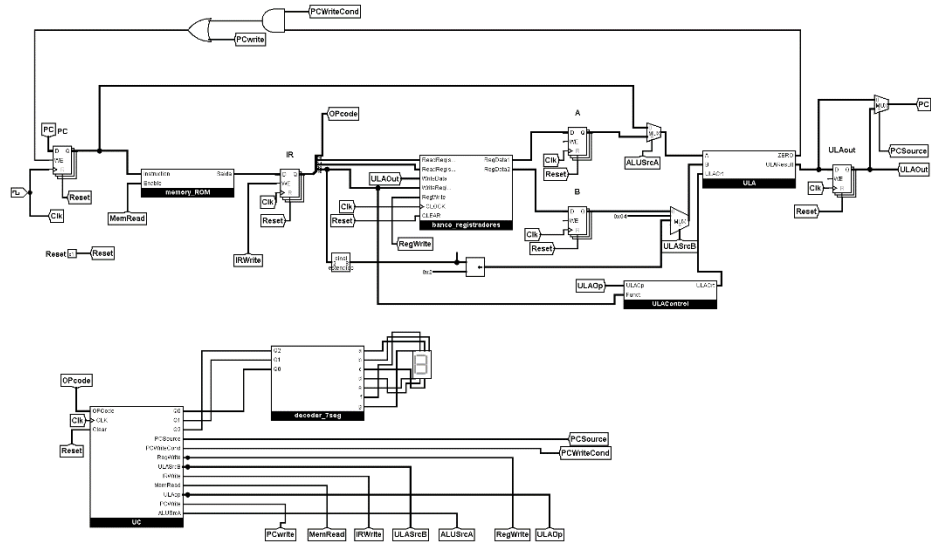


Figura 5: Datapath da arquitetura MIPS de 8 bits multiciclo

2.2.1 UNIDADE DE CONTROLE CABEDA

2.2.1.1 Diagrama de estados

A imagem abaixo mostra o diagrama de estados da unidade de controle cabeada:

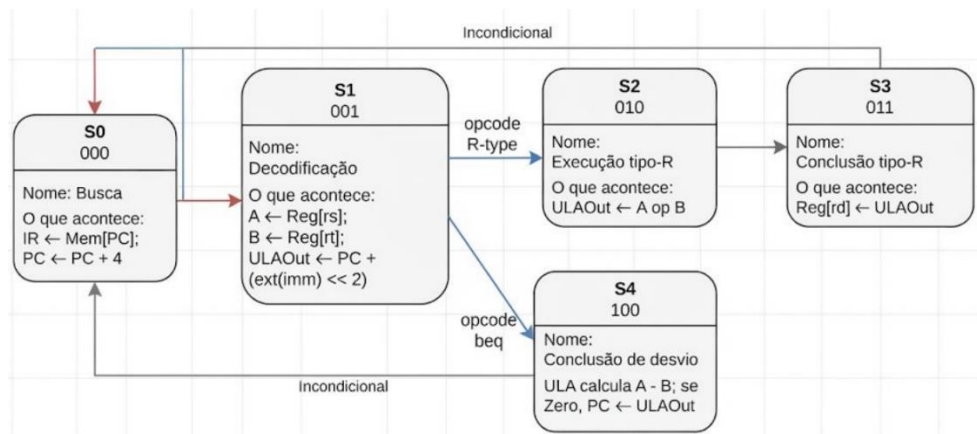


Figura 6: Diagrama de estados

2.2.1.2 Tabela de transição

Estado Atual	Codificação	Entrada	Próximo estado	Saida
S0	000	X	S1	001
S1	001	1(tipo-R)	S2	010
	001	0 (Beq)	S0	000
S2	010	X	S3	011
S3	011	X	S4	100
S4	100	X	S0	000

Tabela 3: Tabela de transição

2.2.1.3 Tabela dos Sinais

Escrita	PCW r	PCWr C	PCSr c	MemR d	IRW r	RegW r	Src A	Src B	ALUO p
S0(Busca)	1	0	0	1	1	0	0	01	00
S1(Decod.)	0	0	X	0	0	0	0	10	00
S2(Exec. Tipo-R)	0	0	X	0	0	0	1	00	10
S3(Conclusão Tipo-R)	0	0	X	0	0	0	1	X	XX
S4(Conclusão beq)	0	1	1	0	0	0	1	00	01

Tabela 4: Tabela verdade dos sinais

2.2.1.4 Mapas de Karnaugh das entradas nos Flip-Flops

D0:

Q2 Opr\ Q0 Q1	00	01	11	10

00	0	0	0	0
01	0	0	0	0
11	0	0	0	0
10	1	0	0	0

D1:

Q2 Opr\ Q0 Q1	00	01	11	10
00	0	1	0	0
01	0	1	0	0
11	0	1	0	0
10	0	1	0	0

D2:

Q2 Opr\ Q0 Q1	00	01	11	10
00	1	1	1	1
01	0	0	0	0
11	0	0	0	0
10	0	0	0	0

2.2.1.4 Equações Booleanas finais

$$D2 = \sim Q2 \cdot \sim Q1 \cdot Q0 \cdot \sim Opr$$

$$D1 = \sim Q2 \cdot \sim Q1 \cdot Q0 \cdot Opr + \sim Q2 \cdot Q1 \cdot \sim Q0$$

$$D0 = \sim Q2 \cdot \sim Q1 \cdot \sim Q0 + \sim Q2 \cdot Q1 \cdot \sim Q0$$

2.2.1.5 Circuito da unidade de controle

O circuito da unidade de controle foi construído a partir de porta AND, OR e NOT, totalizando 13 portas.

A versão microprogramada apresenta mais rapidez de projetar e implementar, pois na versão cabeada, foi necessário preencher a tabela de transição, resolver Mapas de Karnaugh de 4 variáveis para os 3 Flip-Flops e para os sinais de saída, além de fazer a passagem manual de dezenas de fios no Logisim, o que tornou o processo propenso a erros de fiação. Na versão microprogramada, a estrutura física do circuito é genérica e toda a lógica de controle foi configurada apenas criando e editando o arquivo de texto microcodigo.txt, sem necessidade de redesenhar circuitos

Pergunta 3: Se fosse preciso acrescentar uma quinta operação à ULA ou um novo tipo de instrução, quantas equações teriam de ser refeitas na cabeada e quantas microinstruções na microprogramada?

Resposta:

Na versão cabeada o número de estados da FSM passaria de 5 para 6 ou mais. Todas as **3 equações de próximo estado** (D2, D1, D0) e a maioria dos **9 mintermos de saída** teriam que ser totalmente recalculados via Mapas de Karnaugh, exigindo refazer fisicamente as conexões de portas AND/OR no Logisim.

Logo como a versão microprogramada possui uma estrutura genérica, seriam necessários acrescentar 1 ou 2 novas linhas no microcódigo e alterar as constantes de lógica de despacho sem grandes mudanças no circuito.

Pergunta 4: Em que situação de projeto real cada uma das duas abordagens seria preferível?

Resposta:

Pela facilidade de expansão da versão microprogramada, seria preferível a utilização desse método em sistemas embarcados expansíveis onde há facilidade de manutenção, correção de falhas e flexibilidade para adicionar novas instruções.

REFERÊNCIAS

PATTERSON, David A.; HENNESSY, John L. **Organização e projeto de computadores: a interface hardware/software**. 5. ed. Rio de Janeiro: Elsevier, 2017.

HARRIS, David Money; HARRIS, Sarah L. **Projeto digital e arquitetura de computadores**. 2. ed. Imagination Technologies / Elsevier, 2013.

STALLINGS, William. **Arquitetura e organização de computadores**. 10. ed. São Paulo: Pearson Education do Brasil, 2017.